MASTER STUDY GUIDE • ARTIFICIAL INTELLIGENCE

AI Foundations, Intelligent Agents & Task Environments

Complete Course Revision Document based on Lecture Curriculum

1. INTRODUCTION & THE 4 VIEWS OF AI

Core Definition

Artificial Intelligence is concerned with the design of intelligence in artificial devices (term coined by John McCarthy, 1956). AI creates systems that think, learn, reason, and make decisions autonomously.

Dimension	Human-like Standard	Rationality Standard (Ideal)
Thinking	**Thinking Humanly:** Cognitive modeling approach; emulates human thought processes (e.g., General Problem Solver - GPS).	**Thinking Rationally:** "Laws of thought" approach using formal logic, inferences, and syllogisms.
Acting	**Acting Humanly:** Turing Test approach; system performs functions requiring human intelligence without physical contact.	**Acting Rationally:** Rational Agent approach; designs agents acting to achieve the best expected outcome.

Turing Test vs. Total Turing Test

Standard Turing Test

- Natural Language Processing (NLP)
- Knowledge Representation
- Automated Reasoning
- Machine Learning

Total Turing Test (Physical)

- All standard capabilities
- **Computer Vision** (perceive objects)
- **Robotics** (manipulate & move)

2. TAXONOMY OF AI SYSTEMS

Type-1: Capabilities

- **Weak / Narrow AI:** Trained for dedicated single tasks (e.g., Siri, Deep Blue).
- **General AI:** Performs any cognitive task as efficiently as a human (Research phase).
- **Super AI:** Hypothetical level surpassing human intelligence.

Type-2: Functionality

- **Reactive Machines:** No memory/past data (e.g., Deep Blue, AlphaGo).
- **Limited Memory:** Stores short-term past state data (e.g., Self-driving cars).
- **Theory of Mind:** Understood human emotions/beliefs (Future).
- **Self-Awareness:** Sentient, conscious AI (Hypothetical).

3. INTELLIGENT AGENTS & PEAS FRAMEWORK

★ PEAS Representation (Performance, Environment, Actuators, Sensors)

An **Agent** maps percept sequences to actions via an `Agent Function` (implemented by an `Agent Program` running on an architecture).

Agent Type	Performance Measure	Environment	Actuators	Sensors

4. ENVIRONMENT DIMENSIONS

Dimension Pair	Definition & Characteristics	Example Comparison
Fully vs. Partially Observable	Sensors access complete state relevant to action vs. noise, missing data.	Chess (Fully) vs. Poker / Taxi Driving (Partially)
Single vs. Multi-Agent	Agent operates in isolation vs. interacting with competitive/cooperative entities.	Crossword (Single) vs. Chess / Driving (Multi-Agent)
Deterministic vs. Stochastic	Next state strictly determined by current state & action vs. probabilistic outcomes.	Sudoku (Deterministic) vs. Driving / Vacuum world (Stochastic)
Episodic vs. Sequential	Atomic independent choices vs. short-term choices having long-term impact.	Part Scanning (Episodic) vs. Chess / Driving (Sequential)
Static vs. Dynamic vs. Semi-Dynamic	Environment constant during deliberation vs. changing continuously vs. score changing with time.	Crossword (Static) vs. Driving (Dynamic) vs. Timed Chess (Semi-Dynamic)
Discrete vs. Continuous	Finite distinct states, time & actions vs. continuously varying values.	Chess (Discrete) vs. Robot Motion Control (Continuous)
Known vs. Unknown	Agent knows "laws of physics" / rules of environment vs. needs to learn rules.	Solitaire (Known) vs. New Video Game (Unknown)

5. THE FOUR BASIC AGENT ARCHITECTURES

1. Simple Reflex Agent

Selects actions strictly on the **current percept** using condition-action rules (`if condition then action`). Ignores percept history. Requires full observability to prevent infinite loops (randomization can help escape loops).

2. Model-Based Reflex Agent

Handles partial observability by maintaining an internal **state**. Uses a **model** of how the world evolves independently and how the agent's actions affect the world.

3. Goal-Based Agent

Combines internal state & model with explicit **goal information** describing desirable situations. More flexible because decision logic can be modified simply by updating goals.

4. Utility-Based Agent

Uses an internal **utility function** to measure degree of preference/happiness across states. Enables optimal trade-off decisions when goals conflict or carry probabilistic outcomes.

6. WELL-DEFINED PROBLEM FORMULATION

Components of a Search Problem

1. **Initial State:** The state where the search begins.
2. **Actions:** Set of executable actions `ACTIONS(s)` available in state `s`.
3. **Transition Model:** Returns result state `RESULT(s, a)`.
4. **Goal Test:** Determines whether state `s` satisfies goal condition.
5. **Path Cost:** Sum of step costs `c(s, a, s')` leading to node state.

7. UNINFORMED (BLIND) SEARCH STRATEGIES

Uninformed search algorithms explore state space using only problem definitions without heuristic hints regarding goal distance.

1. Breadth-First Search (BFS)

Expands shallowest node first using a **FIFO Queue**. Applies goal test when a node is **generated**.

- Completeness: **Yes** (if branching factor *b* is finite).
- Time & Space Complexity: $O(b^d)$ — Memory overhead is the primary bottleneck.
- Optimality: **Yes** if all step costs are equal.

2. Uniform Cost Search (UCS)

Expands lowest path cost node $g(n)$ using a **Priority Queue**. Applies goal test when a node is **selected for expansion**.

- Completeness & Optimality: **Yes** if step costs $\geq \varepsilon > 0$.
- Complexity: $O(b^{1 + \lfloor C^* / \varepsilon \rfloor})$.

3. Depth-First Search (DFS)

Expands deepest unexpanded node using a **LIFO Queue / Stack**.

- Completeness: **No** in infinite state spaces or spaces with cycles.
- Space Complexity: $O(b \cdot m)$ — Linear space efficiency!

4. Iterative Deepening Search (IDS)

Progressively runs Depth-Limited Search (DLS) with increasing depth limits (0, 1, 2, ...).

- Combines **BFS completeness & optimality $O(b^d)$** with **DFS linear memory $O(b \cdot d)$**. Preferred method for large search spaces with unknown goal depth.

8. MASTER UNINFORMED SEARCH COMPARISON TABLE

Algorithm	Frontier DS	Complete?	Time Complexity	Space Complexity	Optimal?
BFS	FIFO Queue	Yes (if b finite)	$O(b^d)$	$O(b^d)$ [High]	Yes (if unit costs)
UCS	Priority Queue (g)	Yes (step cost > 0)	$O(b^{1 + \lfloor C^* / \varepsilon \rfloor})$	$O(b^{1 + \lfloor C^* / \varepsilon \rfloor})$	Yes
DFS	LIFO Stack	No (in infinite state)	$O(b^m)$	$O(b \cdot m)$ [Low]	No
DLS	LIFO with limit *l*	No (if $l < d$)	$O(b^l)$	$O(b \cdot l)$	No
IDS	LIFO (re-deepening)	Yes	$O(b^d)$	$O(b \cdot d)$ [Optimal]	Yes (if unit costs)
Bidirectional	2 x Frontiers	Yes (if BFS used)	$O(b^{d/2})$	$O(b^{d/2})$	Yes

9. COMPREHENSIVE AI HISTORY TIMELINE & KEY MILESTONES

Year / Era	Key Milestone / Development	Pioneers / Details
1943	First artificial neuron mathematical model	Warren McCulloch & Walter Pitts
1949	Hebbian Learning updating rule introduced	Donald Hebb
1950	"Computing Machinery and Intelligence" & Turing Test	Alan Turing
1955	Logic Theorist (first AI program; proved 38/52 theorems)	Allen Newell & Herbert Simon
1956	Dartmouth Conference — Term "Artificial Intelligence" coined	John McCarthy, Minsky, Shannon, Rochester
1966	ELIZA (first NLP chatbot psychotherapist)	Joseph Weizenbaum
1972	WABOT-1 built in Japan (first humanoid robot)	Waseda University
1974–1980	**First AI Winter**	Severe government funding cuts & slow progress
1980–1987	Rise of Expert Systems & AAAI Stanford Conference	Domain-specific expert rule engines (e.g., XCON)
1987–1993	**Second AI Winter**	Collapse of LISP hardware market & maintenance costs
1997	IBM Deep Blue defeats Garry Kasparov in Chess	IBM Team
2002	Roomba autonomous vacuum cleaner enters homes	iRobot
2005	STANLEY autonomous car wins DARPA Grand Challenge (132 mi)	Sebastian Thrun / Stanford
2011	IBM Watson wins *Jeopardy!* quiz show	IBM NLP & QA System
2014	Eugene Goostman chatbot convinces judges in Turing Test	Vladimir Veselov et al.
2016	AlphaGo beats 18-time world champion Lee Sedol in Go	Google DeepMind

10. QUICK REVISION KEY POINTS FOR EXAM DAY

- ✓ **Goal Test Application:** BFS tests goals upon *generation*; UCS tests goals upon *pop/expansion*.
- ✓ **Memory Efficiency:** IDS achieves BFS optimality while retaining DFS linear memory space $O(bd)$.
- ✓ **Simple Reflex Limitations:** Simple reflex agents fail in partially observable environments unless randomized.
- ✓ **Utility vs Goal:** Goals give binary success states; utility functions give continuous degree of performance.